

Improving Jersey Number Recognition through Data Augmentation and Super-Resolution

Aarav Gosalia*, Qingyun Qian*, Bao Pham*, Raunak Khanna*, Ishaan Singh Yadav*, Hawakulu Abdullahi Dangana*

**Dept. of Computer Science, Mathematics, Physics and Statistics
University of British Columbia
Kelowna, BC, Canada*

Abstract—Accurate jersey number recognition in sports video is a difficult problem because player tracklets contain low-resolution, motion-blurred crops in which digits may be partially occluded. The current state-of-the-art pipeline by Koshkina and Elder uses a fine-tuned PARSeq scene text recognition model and achieves 87.45% top-1 accuracy on the SoccerNet test set. We build directly on this baseline and investigate three targeted improvements: (1) sports-specific data augmentation with corrected fine-tuning, (2) blind super-resolution preprocessing using Real-ESRGAN, and (3) replacing PARSeq with CLIP4STR together with TensorRT inference optimization. Our best configuration—augmentation strategy v1 with fixed fine-tuning at a constant learning rate of 2.5×10^{-6} —raises test accuracy to 87.9%. We find that applying Real-ESRGAN only at inference time collapses accuracy to 45% due to distribution mismatch, but retraining PARSeq on ESRGAN-preprocessed crops recovers performance to approximately 85%. CLIP4STR without fine-tuning reaches only 60.2% due to sensitivity to preprocessing and its full Unicode character set; after fine-tuning on jersey number crops it recovers to 87.5%, matching the baseline but requiring substantially greater setup complexity. TensorRT optimization reduces STR inference time by 31% without changing model behavior. Our results clarify which improvements are viable for this domain and identify distribution alignment as the central challenge when combining super-resolution with a pre-trained recognizer.

Index Terms—jersey number recognition, scene text recognition, data augmentation, super-resolution, Real-ESRGAN, PARSeq, CLIP4STR, TensorRT, SoccerNet

I. INTRODUCTION

The goal of this project is to build a robust system that recognizes a player’s jersey number from short video tracklets, sequences of player thumbnail images extracted over time, from the SoccerNet Jersey Number Recognition dataset [1]. The system must output a single jersey number in the range 1–99, or –1 when no jersey number is visible in the tracklet.

Accurate jersey number recognition matters because team sports video understanding depends heavily on long-term tracking and re-identification of players across many thousands of frames. Since players on the same team wear nearly identical uniforms, the jersey number becomes the key discriminative cue for separating individual identities, particularly in crowded scenes where players cluster and occlude one another.

This task is genuinely challenging in the SoccerNet setting. Player thumbnails are often low-quality, suffering from low

resolution and heavy motion blur arising from fast player movement and broadcast camera panning. Jersey digits may be clearly visible in only a small subset of frames within any given tracklet. Occlusion by other players, sudden changes in body pose, projective distortions from oblique camera angles, and physical folds in the jersey fabric all further reduce legibility. A reliable system must therefore do two things well: identify which frames within a tracklet carry useful information, and aggregate noisy per-frame predictions into a single, confident tracklet-level label.

II. LITERATURE REVIEW

Understanding what has already been attempted—and where prior approaches have fallen short—is essential before proposing any improvements. Our review covers two interconnected areas that together define the state of the art: jersey number recognition in sports video, and scene text recognition (STR), which underlies the most effective modern approaches to the problem.

A. Jersey Number Recognition

Early methods treated jersey number recognition as a standard image classification problem, training convolutional neural networks from scratch on sport-specific labelled datasets. Gerke et al. [2] were among the first to apply this approach to soccer footage. While straightforward to implement, this framing requires a large proprietary labelled dataset and produces a closed-vocabulary classifier: numbers not seen during training simply cannot be predicted, which makes the system fragile in real-world deployment where rosters change and unseen numbers appear regularly.

Liu and Bhanu [3] improved on the basic classification approach by incorporating body pose estimation to automatically locate the jersey region before classification, reducing noise from the background and surrounding body parts. Vats et al. [4] pushed this further with a multi-task learning framework that jointly predicts both the full jersey number and each individual digit, demonstrating that digit-aware supervision consistently outperforms single-task classification. Their approach achieves 89.6% accuracy on a private dataset of ice hockey images. However, both methods inherit the closed-vocabulary limitation of CNN classifiers, and neither dataset has been released publicly, preventing direct comparison.

At the tracklet level, Balaji et al. [5] tackled the temporal aggregation problem directly by developing a keyframe identification module that selects the most informative frames from each tracklet before processing them with a spatio-temporal network. Their method achieves 68.53% on the SoccerNet test set, which was the strongest published tracklet-level result at the time and demonstrated that careful frame selection is critical for reliable number recognition in broadcast footage.

B. Scene Text Recognition as a Solution

A more general solution was proposed by Koshkina and Elder [6], who reframed jersey number recognition as a variant of scene text recognition (STR). Rather than classifying images into a fixed set of known jersey numbers, their system treats the digits as text to be read. This conceptual shift is significant: STR models pre-trained on large synthetic and real-world text datasets can generalize to jersey numbers never seen during fine-tuning, removing the closed-vocabulary fragility of all prior classification-based approaches.

Their pipeline filters illegible frames with a ResNet34 legibility classifier, uses ViTPose to crop the torso region from each remaining frame, and applies PARSeq [7]—a Vision Transformer-based STR model with a permuted autoregressive decoder—to produce a per-frame digit prediction. These per-frame predictions are consolidated into a single tracklet label via a confidence-weighted majority vote, with one-digit predictions down-weighted when two-digit predictions are also present in the tracklet. The full pipeline achieves 91.4% accuracy on hockey images, 87.45% on the SoccerNet test set, and 79.31% on the SoccerNet challenge set [1], surpassing all prior published results by a substantial margin.

Despite these strong results, the pipeline has well-documented failure modes. As shown in Fig. 1, the recognition stage sits at the centre of the pipeline and is the primary source of errors. The authors’ own error analysis reveals that 48% of remaining mistakes involve two-digit numbers being predicted as one-digit numbers, caused by partial occlusion hiding one digit entirely during both training and inference. Low-resolution crops compound this problem: as illustrated in Fig. 2, over 94% of SoccerNet player thumbnails have a height below 64 pixels, giving PARSeq too little visual information to recover digit shapes reliably. The consolidation step attempts to compensate for these issues with manually tuned confidence thresholds, but these thresholds are fragile and do not generalize well across different footage conditions.

Because the Koshkina and Elder framework is both the current state-of-the-art and openly available, it is the natural foundation to build upon. Rather than redesigning the pipeline from scratch, we target these specific, measurable failure modes directly: training robustness to partial digit occlusion through domain-specific data augmentation, resolution recovery through super-resolution preprocessing, and evaluation of a stronger STR backbone alongside inference-time optimization.

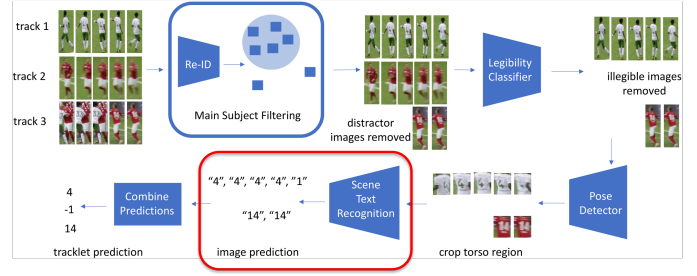


Fig. 1. The tracklet-level jersey number recognition pipeline proposed by Koshkina and Elder [6]. The scene text recognition stage (highlighted) is the primary target of our improvements.

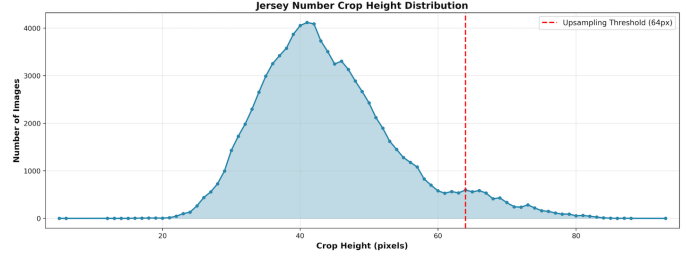


Fig. 2. Distribution of player crop heights in the SoccerNet test set. Over 94% of crops fall below 64 pixels in height.

III. SYSTEM DESIGN

This section describes the baseline pipeline we built upon, the three targeted improvements we implemented, and the hardware and software environment used throughout.

A. Baseline Pipeline

Our starting point is the open-source pipeline of Koshkina and Elder [6]. The system takes a player tracklet as input—a sequence of video frames in which a single player has been detected and cropped by an upstream tracker—and outputs either a single integer jersey number in the range 1–99, or -1 if no legible number can be determined. The pipeline proceeds through five sequential stages, described below.

Stage 1 — Main subject filtering. Centroid-ReID [8] extracts a feature vector for every frame in the tracklet. An isotropic Gaussian is fitted to the full set of feature vectors, and any frame whose vector lies more than $N = 3.5$ standard deviations from the mean is discarded as a distractor (i.e. a frame in which another player has walked in front of the subject). This process is repeated $K = 3$ times. The filtering is entirely unsupervised and requires no per-frame labels.

Stage 2 — Legibility classification. A ResNet34 binary classifier, fine-tuned on pseudo-labels derived from tracklet-level annotations, assigns each remaining frame a legible or illegible label. Frames classified as illegible are discarded. A tracklet is considered illegible overall if no frame passes this stage.

Stage 3 — Torso cropping. ViTPose estimates 17 body keypoints for each legible frame. A bounding box is defined by the shoulder and hip keypoints and padded by 5 pixels on

the left, right, and bottom. This crop isolates the torso region where the jersey number appears and is the direct input to the recognition stage.

Stage 4 — Scene text recognition. Fine-tuned PARSeq [7] reads each torso crop and produces a predicted digit string along with a per-character confidence score.

Stage 5 — Prediction consolidation. Per-frame predictions are aggregated into a single tracklet-level label via a confidence-weighted majority vote. When one-digit and two-digit predictions both appear within the same tracklet, one-digit votes are down-weighted to account for the known tendency for partially occluded two-digit numbers to be read as single digits. If the aggregate confidence falls below a fixed threshold, the tracklet is labelled -1 .

B. Improvement 1: Data Augmentation and Fine-Tuning

1) *Motivation:* The baseline PARSeq model was fine-tuned on weakly labelled jersey number crops that are relatively clean and fully visible. It therefore has limited robustness to the partial occlusion and broadcast-specific degradations that cause the majority of errors at test time. We developed two complementary augmentation strategies and, in the process, identified and corrected a critical bug in the fine-tuning procedure itself.

2) *Corrected Fine-Tuning Procedure:* The original repository resumes fine-tuning via PyTorch Lightning’s `ckpt_path` argument, which restores not only the model weights but also the full optimizer state, epoch counter, and OneCycleLR learning rate schedule. OneCycleLR is a single-cycle policy: the learning rate rises to a peak of approximately 7×10^{-4} around 30% of the way through the total training steps, then decays continuously to a near-zero floor by the final step. After the original 25-epoch pre-training run the scheduler is therefore sitting at the very end of its cycle. When `ckpt_path` is used to resume for fine-tuning, PyTorch Lightning also restores the epoch counter to epoch 24, so the trainer considers training almost complete. The scheduler’s internal step count is then misaligned with the new one-epoch fine-tuning objective and the new dataloader length, producing erratic learning rate behaviour and preventing any meaningful weight updates from occurring.

The corrected procedure makes three changes. First, model weights are loaded in isolation using a custom `pretrained` flag that ignores all optimizer and scheduler state. Second, OneCycleLR is replaced with a constant learning rate of 2.5×10^{-6} , which is 280 times lower than the original schedule peak and prevents destructive updates to the pre-trained feature representations. Third, training is limited to exactly one epoch; additional epochs cause overfitting on the weak pseudo-labels and degrade performance. This fix is a prerequisite for any augmentation strategy to yield an improvement.

3) *Augmentation Strategy v1:* Strategy v1 applies three augmentations to each training sample. Each augmentation is applied independently, meaning any combination of the three can be active on the same image simultaneously. Five augmented copies are generated per original crop, expanding



Fig. 3. Augmentation strategy v1 applied to a torso crop. From left to right: original, Gaussian blur and noise, perspective warp, and rectangular masking. Augmentations are applied independently and can stack on the same image.

the training set to approximately six times its original size before shuffling.

- **Gaussian blur and additive noise** (applied with $p = 0.5$). A Gaussian blur with radius sampled uniformly from $[0.3, 0.8]$ pixels is applied, followed by additive Gaussian noise with standard deviation sampled from $[3, 12]$ intensity units. This simulates the general video degradation present in broadcast footage.
- **Perspective warp** (applied with $p = 0.5$). Each of the four corners of the crop is displaced by a random offset scaled to at most 15% of the half-image dimension in both axes, producing a plausible projective distortion. Horizontal flipping is explicitly excluded to avoid swapping visually similar digit pairs such as 6 and 9.
- **Rectangular masking** (applied with $p = 0.35$). Between one and two rectangles, each with a width of 5–15% of the crop width and a height of 10–25% of the crop height, are placed at uniformly random positions and filled with the image’s mean colour. This directly simulates one digit being hidden by an opposing player’s arm or body.

Fig. 3 illustrates the three augmentations applied to the same crop.

4) *Augmentation Strategy v2:* Strategy v2 is motivated by the systematic STR augmentation study of Atienza [9] (STRAug), which catalogued which degradation types most commonly cause recognition failures and which augmentations best address them. A central finding of that work is that PARSeq’s internal RandAugment pipeline already applies 15 standard augmentations at training time—including Gaussian blur, brightness and contrast jitter, shear, and rotation—meaning that replicating these effects provides no additional benefit. Strategy v2 therefore targets only the three domain specific degradations that PARSeq’s default pipeline does not cover for broadcast sports video.

- **Motion blur** (applied with $p = 0.4$). A 3×3 directional convolution kernel is applied in one of three directions: horizontal, vertical, or diagonal, chosen uniformly at random. This simulates the directional blur produced by camera panning and fast player movement, which differs from the isotropic Gaussian blur already present in RandAugment.
- **Pixelation** (applied with $p = 0.3$). The crop is downsampled by a factor of 2 using nearest-neighbour interpolation



Fig. 4. Augmentation strategy v2. From left to right: original, motion blur, pixelation, and JPEG compression. Motion blur and pixelation are mutually exclusive per image; JPEG compression can stack on top of either.

and then upsampled back to its original dimensions, also using nearest-neighbour interpolation. This reproduces the blocky low-resolution appearance of crops taken from distant-camera frames.

- **JPEG compression** (applied with $p = 0.3$). The image is re-encoded as JPEG at a quality level sampled uniformly from $[30, 70]$ and then reloaded. This introduces the block artefacts characteristic of broadcast video compression.

Motion blur and pixelation are applied *mutually exclusively*: at most one spatial degradation is applied per image to prevent generating entirely unreadable crops. JPEG compression is independent and may stack on top of either. Fig. 4 illustrates strategy v2.

C. Improvement 2: Super-Resolution with Real-ESRGAN

1) *Motivation*: As shown in Fig. 2, over 94% of SoccerNet player crops have a height below 64 pixels. At this scale individual digit strokes occupy only a handful of pixels, making reliable recognition difficult even for a well-trained model. We investigate whether upscaling crops with Real-ESRGAN [10] prior to recognition can recover sufficient detail for PARSeq to read digits accurately.

Real-ESRGAN is a blind super-resolution model trained entirely on synthetically degraded images designed to approximate real-world degradation processes. It applies a $4\times$ upscale using the `RealESRGAN_x4plus.pth` pretrained weights. Unlike bicubic interpolation, it reconstructs plausible high-frequency detail rather than simply smoothing pixel values, producing sharper edges and more defined structures.

2) *Implementation: Attempt 1 (Inference Only)*: In the first configuration, Real-ESRGAN is applied to every torso crop at inference time immediately before it is passed to PARSeq. The PARSeq weights remain unchanged from the baseline. This is the simplest possible integration: no retraining is required and the change is confined to a single preprocessing step controlled by a `--use_esrgan` flag in `str.py`.

3) *Implementation: Attempt 2 (Training and Inference)*: The second configuration applies Real-ESRGAN consistently at both training and inference. The entire training LMDB is preprocessed with `esrgan_lmdb.py`, which reads each image from the source LMDB, applies the $4\times$ Real-ESRGAN upscale, and writes the result to a new `lmdb_esrgan` database. PARSeq is then fine-tuned on this enhanced dataset, with the best checkpoint selected by validation accuracy.



Fig. 5. A SoccerNet torso crop before (left) and after (right) Real-ESRGAN $4\times$ upscaling. The upscaled image recovers sharper edges but introduces hallucinated texture, the implications of which are discussed in Section V.

At inference, Real-ESRGAN is applied to all crops before recognition, matching the distribution the model was trained on. A separate `esrgan` conda environment is used for the preprocessing step to manage the `numpy<2` and `torchvision<0.16` version constraints required by the Real-ESRGAN library.

Fig. 5 shows a representative crop before and after Real-ESRGAN upscaling.

4) *Simple Gated Bicubic Upscale*: As a lightweight reference point, we also implement a gated bicubic upscale: any crop whose height falls below 60 pixels is upscaled by a factor of $2\text{--}3\times$ using OpenCV’s bicubic interpolation (`cv2.INTER_CUBIC`) before being passed to the unmodified baseline PARSeq model. No retraining is required. This serves as a low-cost comparison to assess how much of any gain from Real-ESRGAN comes simply from providing more pixels to the recognizer versus the content of the reconstructed detail.

D. Improvement 3: CLIP4STR and TensorRT Optimization

1) *CLIP4STR as an Alternative STR Backbone*: CLIP4STR [11] replaces PARSeq’s ViT encoder with CLIP’s ViT-B/16 backbone [12], which was pre-trained on 400 million image-text pairs. A cross-modal decoder branch takes the initial visual prediction, passes it through CLIP’s frozen text encoder, and uses the resulting language-guided features to refine the final output. The architecture is shown in Fig. 6.

CLIP4STR is integrated as a drop-in replacement for the PARSeq inference step, using the publicly available implementation at github.com/VamosC/CLIP4STR. The same weak pseudo-label fine-tuning strategy as the baseline is applied: the legibility classifier identifies legible frames within training tracklets, the identified crops are used as training samples labelled with their tracklet-level ground truth, and the model is fine-tuned on this dataset.

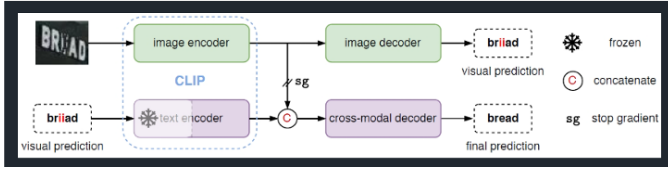


Fig. 6. CLIP4STR architecture [11]. The visual prediction from the image decoder is passed through CLIP’s frozen text encoder in a cross-modal decoder, producing a refined final prediction.

TABLE I
HARDWARE AND SOFTWARE ENVIRONMENT.

Component	Details
GPU (augmentation)	NVIDIA RTX A4000
GPU (TensorRT)	NVIDIA RTX 2060 SUPER
Framework	PyTorch 2.8.0, CUDA
STR model	PARSeq [7], CLIP4STR [11]
Super-resolution	Real-ESRGAN x4plus [10]
Pose estimation	ViTPose [13]
Re-ID	Centroid-ReID [8]
Env. management	conda (5 isolated environments)
Dataset	SoccerNet JNR [1]

2) *TensorRT Inference Optimization*: TensorRT is NVIDIA’s inference compilation and optimization SDK. It accepts a trained model, applies layer fusion, kernel auto-tuning, and optional reduced-precision execution (FP16), and produces a compiled inference engine that runs on the target GPU hardware. We apply TensorRT to the PARSeq STR step using `torch.compile` with the TensorRT backend under PyTorch 2.8.0. The optimization is hardware-specific: the compiled engine is generated and benchmarked on an RTX 2060 SUPER GPU. No changes are made to model weights or to any other pipeline stage. The optimization is measured by comparing wall-clock inference time on the full SoccerNet test set between the native PyTorch baseline and the TensorRT-compiled model on the same hardware.

E. Hardware and Software Environment

Table I summarizes the hardware and software used across experiments. The augmentation and baseline replication experiments were run on a Windows machine with an RTX A4000 GPU. TensorRT experiments were run on a separate machine with an RTX 2060 SUPER. Environment management uses conda, with separate environments for each pipeline stage (`jersey`, `centroids`, `vitpose`, `parseq2`, `esrgan`) to isolate conflicting dependency requirements.

All three improvements were implemented and evaluated independently. No two improvements were active simultaneously, ensuring that each result reported in Section IV reflects the isolated contribution of a single change to the pipeline.

IV. RESULTS

A. Baseline Replication

Before evaluating any improvements, we replicated the Koshkina and Elder pipeline [6] on the SoccerNet test set to

1212 1211	Total number of tracklets: 1211, correct: 1053, accuracy: 86.95293146160198%
1211 1211	Total number of tracklets: 1211, correct: 1055, accuracy: 87.11808422791081%

Fig. 7. Replication results on the SoccerNet test set. Top: 86.95% (1053/1211 correct). Bottom: 87.12% (1055/1211 correct). We use 87.0% as our working baseline, consistent with the 87.45% reported in [6] given minor environment variation.

TABLE II
AUGMENTATION AND FINE-TUNING ABLATION ON THE SOCCERNET TEST SET.

Configuration	Test Accuracy
Baseline (no augmentation)	87.0%
v1 + broken fine-tuning	82.2%
v2 + broken fine-tuning	80.3%
v2 + fixed fine-tuning ($LR = 1 \times 10^{-5}$)	86.1%
v2 + fixed fine-tuning ($LR = 5 \times 10^{-6}$)	87.3%
v2 + fixed fine-tuning ($LR = 2.5 \times 10^{-6}$)	87.7%
v1 + fixed fine-tuning ($LR = 2.5 \times 10^{-6}$)	87.9%

establish a locally verified baseline. The authors report 87.45% accuracy on this set. As shown in Fig. 7, our replication runs produced 86.95% and 87.12% across two independent runs on the same hardware, with variation attributable to differences in environment and random seed. We treat 87.0% as our working baseline for all subsequent comparisons.

B. Data Augmentation Results

Table II reports test accuracy across all augmentation and fine-tuning configurations. The most critical finding is the effect of the fine-tuning bug: both strategy v1 and strategy v2 hurt accuracy when the broken `ckpt_path` procedure is used, dropping to 82.2% and 80.3% respectively—well below baseline. This confirms that no augmentation benefit is observable until the fine-tuning procedure is corrected.

Once the fix is applied, both strategies recover and surpass baseline. Learning rate sensitivity is clear: 1×10^{-5} is too high, producing residual forgetting at 86.1%, while 2.5×10^{-6} is optimal. Strategy v1 with this learning rate achieves the best result of **87.9%**, a 0.9 percentage point improvement over our 87.0% baseline. Strategy v2 peaks at 87.7% under the same conditions.

Fig. 8 shows the terminal output confirming the best augmentation result.

C. Super-Resolution Results

Applying Real-ESRGAN only at inference time causes accuracy to collapse to approximately 45%, entirely due to the distribution mismatch between ESRGAN-enhanced crops and the training distribution PARSeq was fine-tuned on. Retraining PARSeq on ESRGAN-preprocessed crops resolves the mismatch and recovers accuracy to approximately 85%, but this remains 2 percentage points below the baseline. The distribution is successfully aligned, but Real-ESRGAN’s general-purpose hallucinated texture introduces digit stroke

```
Total number of tracklets: 1211, correct: 1065, accuracy: 87.943848059455%
Task 'Evaluate accuracy' took: 0.0089 s
```

Fig. 8. Evaluation output for the best augmentation configuration: strategy v1 with fixed fine-tuning at $LR = 2.5 \times 10^{-6}$, achieving 87.9% on the SoccerNet test set.

TABLE III
COMPARISON OF ALL CONFIGURATIONS AGAINST THE BASELINE ON THE SOCCERNET TEST SET.

Method	Test Accuracy
Koshkina & Elder [6] (published)	87.45%
Our replication (baseline)	87.0%
Ours: Aug v1 + fixed fine-tuning (best)	87.9%
Ours: Aug v2 + fixed fine-tuning	87.7%
Ours: Bicubic upscale (gated)	87.2%
Ours: ESRGAN inference only	~45.0%
Ours: ESRGAN + retrained PARSeq	~85.0%
Ours: CLIP4STR (no fine-tuning)	60.2%
Ours: CLIP4STR (after fine-tuning)	87.5%

artefacts that a text-specific super-resolution model would be less likely to produce.

The simple gated bicubic upscale achieves 87.2%, a small improvement over baseline at negligible cost and without requiring any retraining, confirming that even modest resolution recovery helps the recognizer on the smallest crops.

D. CLIP4STR and TensorRT Results

Without fine-tuning, CLIP4STR achieves only 60.2%. As described in Section III, this is primarily due to its full Unicode character set diluting prediction confidence across digit classes, and a preprocessing mismatch between our 32×128 crop format and CLIP’s expected 224×224 input. After fine-tuning on jersey number crops, CLIP4STR recovers to **87.5%**, matching the published baseline but requiring substantially greater setup complexity than the augmentation approach.

TensorRT reduces STR step wall-clock time from 1263.85 s to 868.27 s on the SoccerNet test set, a reduction of 31%, with no meaningful change in accuracy. It is the only improvement among the three that directly addresses pipeline latency rather than recognition accuracy.

E. Comparison with Baseline

Table III summarises all configurations against our working baseline and the published Koshkina and Elder result. Our best configuration, augmentation strategy v1 with fixed fine-tuning, exceeds both.

V. DISCUSSION

A. Optimizer State Corruption Masked All Augmentation Results

The fine-tuning bug demonstrates that even loading a correct checkpoint can silently destroy performance if the optimizer state is restored alongside model weights. The misaligned OneCycleLR schedule caused the trainer to believe training was nearly complete, producing erratic learning rate behaviour

that prevented any meaningful adaptation to the new training data. The consequence was that augmentation appeared harmful in every early experiment, leading to results as low as 80.3%, until the root cause was identified. The fix—weights-only loading with a constant learning rate of 2.5×10^{-6} for one epoch—is straightforward in retrospect but non-obvious, undocumented in the original repository, and would have invalidated any conclusion drawn from the augmentation experiments had it not been found.

Once the fine-tuning procedure was corrected, both augmentation strategies improved over baseline, confirming the original hypothesis that exposing PARSeq to partially masked and degraded crops during training improves its robustness to the same conditions at inference. The gain is modest at 0.9 percentage points for the best configuration, corresponding to approximately 11 additional correct tracklet predictions on the 1,211-tracklet test set. This is consistent with what a single epoch of fine-tuning on a relatively small weakly-labelled dataset can reasonably achieve. The learning rate sensitivity observed in Table II further confirms that the pre-trained representations are fragile: 1×10^{-5} is sufficient to cause measurable forgetting even in a single epoch, while 2.5×10^{-6} sits in a narrow effective range that updates only the most task-relevant features.

B. Augmentation v1 Outperforms v2 Despite Stronger Motivation

An unexpected finding is that strategy v1 outperforms strategy v2 (87.9% vs. 87.7%), despite v2 being more carefully motivated by the STR augmentation literature [9]. The likely explanation is the rectangular masking in v1. While it was designed to simulate partial digit occlusion—the dominant error mode responsible for 48% of baseline errors—rectangular masking is also the most aggressive augmentation in the set. At $p = 0.35$ it risks occasionally obscuring so much of a digit that the label becomes ambiguous or misleading, which should in principle hurt rather than help. That v1 still comes out ahead of v2 suggests that the benefit of directly simulating the target failure mode outweighs the noise introduced by occasionally over-masking, and that the occlusion problem is severe enough that even imperfect occlusion simulation provides a stronger training signal than the broadcast-specific degradations targeted by v2.

C. Distribution Alignment is the Central Challenge for Super-Resolution

Real-ESRGAN exposed a different but equally important principle: every preprocessing step in the pipeline must preserve the downstream model’s expected input distribution. Applying a general-purpose super-resolution model only at inference creates a distribution shift that PARSeq cannot compensate for, even though the ESRGAN output looks sharper and more legible to the human eye. The confidence collapse—per-digit confidence dropping from approximately 0.90 to 0.55, compounding to a frame-level score of roughly 0.30 for a two-digit number—caused the majority of tracklets

to fall below the illegibility threshold and be labelled -1 rather than contributing a vote. The 45% accuracy figure is therefore not a recognition failure; it is almost entirely a consolidation failure triggered by the distribution mismatch. Retraining PARSeq on ESRGAN-preprocessed crops resolved the mismatch and recovered accuracy to approximately 85%, confirming the diagnosis. The residual 2 percentage point gap below baseline is attributable to Real-ESRGAN’s general-purpose hallucination model introducing texture patterns that are plausible for natural images but geometrically incorrect for digit strokes, a problem that a text-specific super-resolution model would be less likely to produce.

D. Simple Upscaling as a Low-Cost Partial Solution

The gated bicubic upscale achieves 87.2%, a 0.2 percentage point improvement over the 87.0% baseline. While this is technically above baseline, the gain is negligible in practical terms—it corresponds to roughly two additional correct predictions across the entire test set—and falls within the range of run-to-run variation observed during replication. Its value is primarily as a reference point: it demonstrates that simply providing more pixels to the recognizer, without any content-aware reconstruction, does offer a marginal benefit on the smallest crops. This suggests that resolution is a real limiting factor, and that a text-specific super-resolution model that avoids the hallucination problems of Real-ESRGAN could potentially translate this observation into a more meaningful accuracy improvement.

E. CLIP4STR Requires a Dedicated Integration

The 60.2% accuracy obtained from CLIP4STR without fine-tuning should not be interpreted as a ceiling on what the model can achieve for this task. Two structural mismatches explain the poor out-of-the-box performance. First, CLIP4STR operates over the full Unicode character set, meaning that when predicting a two-digit jersey number the model must assign probability mass across hundreds of character bins rather than ten. Second, CLIP’s ViT-B/16 encoder expects 224×224 inputs with normalization specific to its pre-training distribution, while our pipeline supplies 32×128 crops normalized for PARSeq. Neither of these is a fundamental limitation of the model; both are integration problems. After fine-tuning on jersey number crops, CLIP4STR recovers to 87.5%, matching the published baseline, which confirms that the model is capable of learning the task when given appropriate supervision. However, this result also shows that CLIP4STR does not improve over a well-tuned PARSeq under our current integration, and the substantially greater setup complexity means it offers no practical advantage at this stage.

F. TensorRT as a Free Performance Boost

TensorRT optimization reduced the STR inference step from 1263.85 s to 868.27 s on the full SoccerNet test set, a 31% reduction, with no change to model weights or pipeline behaviour. Unlike the other three improvements, TensorRT

requires no new training data, no modified fine-tuning procedure, and no changes to any other pipeline component. It is hardware-specific—the compiled engine is tied to the RTX 2060 SUPER it was generated on—but on any supported NVIDIA GPU it represents a genuinely free gain in throughput. For a pipeline that takes several hours to run end-to-end on a full dataset, a 31% reduction in a compute-intensive step has meaningful practical value for any deployment scenario requiring repeated evaluation.

VI. FUTURE WORK

Based on our findings, we identify four concrete directions for future work.

Combining augmentation strategies. Strategies v1 and v2 target different degradation types (occlusion and broadcast video artifacts, respectively). A combined strategy that applies rectangular masking (v1) alongside motion blur and JPEG compression (v2) may improve robustness to both failure modes simultaneously.

Text-specific super-resolution and deeper fine-tuning. Real-ESRGAN is a general natural image super-resolution model. Models specifically designed to sharpen text strokes—such as those developed for license plate recognition or low-resolution document OCR—are likely to produce higher-fidelity digit reconstructions without introducing the hallucinated texture that harmed our results. Beyond the choice of super-resolution model, the fine-tuning of PARSeq on ESRGAN-preprocessed crops in our experiments was relatively shallow. A more thorough fine-tuning procedure—with careful learning rate scheduling across multiple epochs, or applied to an alternative STR backbone better suited to the enhanced input distribution—may close the remaining gap between the ESRGAN pipeline and the baseline. Both directions should be evaluated in combination with the augmented fine-tuning strategy to determine whether resolution recovery and occlusion robustness are complementary gains.

Proper CLIP4STR integration. A purpose-built CLIP4STR pipeline requires adapting crop resizing to 224×224 , restricting the character vocabulary to digits 0–9, and potentially modifying the confidence computation for the consolidation step. Given CLIP4STR’s documented advantages on occluded and irregular text [11], a proper integration may yield accuracy improvements beyond what augmentation alone can achieve.

Broader TensorRT coverage. TensorRT was applied only to the PARSeq STR step. Extending it to the legibility classifier (ResNet34) and the Centroid-ReID feature extraction step would further reduce end-to-end latency, making the pipeline more practical for live broadcast processing.

VII. CONCLUSION

This work investigated three targeted improvements to the state-of-the-art jersey number recognition pipeline of Koshkina and Elder [6] on the SoccerNet dataset. Our starting point was a local replication of the baseline at approximately 87.0%, consistent with the published 87.45%.

The most impactful finding was not a new model or technique, but the discovery and correction of a broken fine-tuning procedure in the original repository. The use of `ckpt_path` to resume training restored the full optimizer state and epoch counter, misaligning the OneCycleLR schedule with the new fine-tuning objective and preventing any weight updates from occurring. Once corrected—through weights-only loading, a constant learning rate of 2.5×10^{-6} , and a single training epoch—domain-specific data augmentation improved test accuracy from 87.0% to **87.9%**. Augmentation strategy v1, which directly simulates partial digit occlusion through rectangular masking, outperformed the literature-backed strategy v2, suggesting that targeting the dominant failure mode directly provides a stronger training signal than covering degradations not already handled by PARSeq’s internal augmentation pipeline.

Real-ESRGAN super-resolution demonstrated that distribution alignment is a prerequisite for any preprocessing change to succeed. Applied only at inference, it collapsed accuracy to 45% through a confidence failure in the consolidation step. Applied consistently at both training and inference, it recovered to approximately 85%—still below baseline—due to hallucinated texture ill-suited to digit stroke reconstruction. A simple gated bicubic upscale achieved 87.2% at negligible cost, confirming that resolution is a genuine bottleneck but that a general-purpose super-resolution model may not be the right tool to address it.

CLIP4STR without fine-tuning reached only 60.2% due to Unicode character set breadth and preprocessing mismatch, but recovered to 87.5% after fine-tuning, matching the baseline without improving upon it. TensorRT optimization reduced STR inference time by 31% with no accuracy cost, representing the only improvement that is both free to apply and immediately deployable.

Taken together, these results show that careful fine-tuning procedure design and domain-specific occlusion augmentation are the most reliable paths to accuracy improvement on this task under the current pipeline architecture. Distribution alignment and backbone-specific integration requirements are the central obstacles for super-resolution and alternative STR models respectively, and both remain open problems for future work.

ACKNOWLEDGMENT

The authors acknowledge the original work of Koshkina and Elder [6] for providing the open-source pipeline and pre-trained models on which all our experiments are built.

CODE AVAILABILITY

All code developed for this project, including the augmentation scripts, corrected fine-tuning procedure, Real-ESRGAN integration, and TensorRT optimization, is available at: <https://github.com/TriangleMesh/jersey-number-pipeline>

REFERENCES

- [1] A. Cioppa, A. Deliège, S. Giancola, B. Ghanem, and M. V. Droogebroek, “Scaling up SoccerNet with multi-view spatial localization and re-identification,” *Scientific Data*, vol. 9, 2022.
- [2] S. Gerke, K. Muller, and R. Schafer, “Soccer jersey number recognition using convolutional neural networks,” in *Proceedings of the IEEE ICCV Workshops*, 2015, pp. 17–24.
- [3] H. Liu and B. Bhanu, “Pose-guided R-CNN for jersey number recognition in sports,” in *Proceedings of the IEEE/CVF CVPR Workshops*, 2019.
- [4] K. Vats, M. Fani, D. A. Clausi, and J. Zelek, “Multi-task learning for jersey number recognition in ice hockey,” in *Proceedings of the 4th International Workshop on Multimedia Content Analysis in Sports (MMSports)*, 2021, pp. 11–15.
- [5] B. Balaji, J. Bright, H. Prakash, Y. Chen, D. A. Clausi, and J. Zelek, “Jersey number recognition using keyframe identification from low-resolution broadcast videos,” in *Proceedings of the 6th International Workshop on Multimedia Content Analysis in Sports (MMSports)*, 2023, pp. 123–130.
- [6] M. Koshkina and J. H. Elder, “A general framework for jersey number recognition in sports video,” in *Proceedings of the IEEE/CVF CVPR Workshops (CVSports)*, 2024, pp. 3235–3244.
- [7] D. Bautista and R. Atienza, “Scene text recognition with permuted autoregressive sequence models,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022, pp. 178–196.
- [8] M. Wiecek, B. Rychalska, and J. Dabrowski, “On the unreasonable effectiveness of centroids in image retrieval,” in *Proceedings of the International Conference on Neural Information Processing (ICONIP)*, 2021, pp. 212–223.
- [9] R. Atienza, “Data augmentation for scene text recognition,” in *Proceedings of the IEEE/CVF ICCV Workshops (ICCVW)*, 2021, pp. 1561–1570.
- [10] X. Wang, L. Xie, C. Dong, and Y. Shan, “Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data,” in *Proceedings of the IEEE/CVF ICCV Workshops*, 2021.
- [11] S. Zhao, R. Quan, L. Zhu, and Y. Yang, “CLIP4STR: A simple baseline for scene text recognition with pre-trained vision-language model,” *IEEE Transactions on Image Processing*, vol. 33, pp. 6893–6904, 2024.
- [12] A. Radford *et al.*, “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.
- [13] Y. Xu, J. Zhang, Q. Zhang, and D. Tao, “ViTPose: Simple vision transformer baselines for human pose estimation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.